Phase 1 Implementation Report

Operating Systems Fall 2025

Khaleeqa Aasiyah Garrett

Kag9691 | Custom Shell Implementation

Architecture and Design

High-Level System Architecture

For this project, I designed MyShell using a modular architecture. I wanted each part of the shell to be separated into its own file so that the code stays organized, easy to read, and easier to expand in later phases.

The structure of the project looks like this:

```
MyShell Architecture
├── main.c       - Main program loop and control flow
├── input.c      - User input handling and prompt display
├── parser.c     - Command parsing and validation
├── executor.c   - Process execution and pipeline management
├── memory.c     - Dynamic memory management
├── utils.c      - Utility functions and error handling
└── shell.h      - Common headers and data structures
```

- **main.c**: Runs the main loop of the shell, repeatedly showing the prompt and calling the parser/executor.
- **input.c**: Reads user input and displays the shell prompt.
- **parser.c**: Breaks down commands into arguments, redirections, and pipes. Validates syntax.
- **executor.c**: Runs commands by creating processes, setting up pipes, and handling redirection.
- **memory.c**: Manages dynamic memory to avoid leaks.
- **utils.c**: Holds helper functions and error handling routines.
- **shell.h**: Defines the key data structures so that all files share the same definitions.

This design made the code cleaner and allowed me to focus on one part of the system at a time.

Key Data Structures and Custom Types

Dynamic Arrays are used for elements whose size varies during execution. For example, command arguments are stored in a dynamically allocated array of string pointers:

```
// parser.c (~line 85)
cmd->args = malloc((token_count + 1) * sizeof(char*));
for (int i = 0; i < token_count; i++) {
    cmd->args[arg_index++] = strdup(tokens[i]);
}
cmd->args[arg_index] = NULL; // NULL-terminated for execvp()
```

Each element points to a dynamically allocated string. A dynamic array is necessary because commands can have different numbers of arguments, e.g., `ls` versus `ls -la -h`. Using a dynamic array also ensures compatibility with `execvp()`, which expects a NULL-terminated argument list.

Pipelines are represented as dynamic arrays of `Command` structures:

```
// parser.c (~line 20)
pipeline->commands = malloc(MAX_COMMANDS * sizeof(Command));
```

This allows the shell to store a sequence of commands connected by pipes. Using a dynamic array keeps all commands contiguous in memory, simplifying sequential access during parsing and execution, while still allowing variable-length pipelines.

Fixed arrays are used where the size is known at runtime and does not change, such as pipe file descriptors:

```
// executor.c (~line 17)
int pipes[pipeline->command_count - 1][2];
```

Each row stores the read and write file descriptors for a pipe between commands. Fixed arrays simplify indexing and are automatically cleaned up when the function exits, reducing the risk of leaks. Similarly, process IDs are stored in a fixed array:

```
// executor.c (~line 18)
pid_t pids[pipeline->command_count];
```

This allows tracking child processes for proper synchronization using `waitpid()`.

Temporary token arrays and **dynamic strings** are also used. Tokens are stored in a stack-allocated fixed array during parsing to simplify tokenization:

```
// parser.c (~line 60)
char *tokens[MAX_TOKENS];
```

The user's input is stored in a dynamic buffer:

```
// input.c (~line 11)
char *input = malloc(MAX_INPUT_SIZE);
```

File paths for redirection are stored using `strdup()` to create independent copies for each command.

Stack usage manages local variables, function return addresses, and recursive cleanup. For example, `free_pipeline()` and `free_command()` use stack frames to safely deallocate nested dynamic memory:

```
// memory.c
void free_pipeline(Pipeline *pipeline) {
    for (int i = 0; i < pipeline->command_count; i++) {
        free_command(&pipeline->commands[i]);
    }
}
```

Custom Data Types

```
typedef struct {
    char **args;          // Command arguments
    char *input_file;     // Input redirection
    char *output_file;    // Output redirection
    char *error_file;     // Error redirection
    int arg_count;        // Number of arguments
} Command;

typedef struct {
    Command *commands;    // Array of commands
    int command_count;    // Total commands in pipeline
} Pipeline;
```

Each `Command` holds all information needed to execute a single command, while a `Pipeline` represents one or more commands connected by pipes. Even a single command is treated as a pipeline to standardize execution logic.

Design Decisions

- **Modular Design:** Separating responsibilities into different files improves readability and maintainability.
- **Dynamic vs Fixed Memory:** Dynamic arrays handle variable-length commands and arguments, while fixed arrays simplify pipe and PID management.
- **Pipeline-Centric Model:** Treating all commands as pipelines ensures consistent execution logic.
- **Early Error Validation:** Syntax and file errors are detected during parsing rather than at runtime.
- **Process Management:** Each command runs in a separate process using `fork()`, with proper redirection using `dup2()` and careful cleanup to prevent resource leaks.

Implementation Highlights

1. Command Parsing (`parser.c`)

The parser is responsible for interpreting the user input, breaking it into commands, arguments, and handling redirections and pipelines. First, the input is tokenized using `strtok_r()` to handle spaces and tabs efficiently:

```
char *token;
char *saveptr;
token = strtok_r(input, " \t\n", &saveptr);
while (token != NULL) {
    tokens[token_count++] = strdup(token);
    token = strtok_r(NULL, " \t\n", &saveptr);
}
```

This approach ensures that multiple spaces or tabs between arguments do not produce empty tokens, allowing commands like `"ls -la /home"` to be parsed correctly.

After tokenization, each command is checked for validity. Empty commands or trailing pipes are caught early to prevent execution errors:

```
if (token_count == 0) {
    fprintf(stderr, "Error: Empty command.\n");
    return NULL;
}
if (strcmp(tokens[token_count - 1], "|") == 0) {
    fprintf(stderr, "Error: Trailing pipe detected.\n");
    return NULL;
}
```

Redirections are also validated during parsing to ensure filenames are provided:

```
if (strcmp(tokens[i], "<") == 0) {
    if (i + 1 >= token_count) {
        fprintf(stderr, "Error: Missing input file for redirection.\n");
        return NULL;
    }
    cmd->input_file = strdup(tokens[++i]);
}
```

This method ensures that all potential user input errors are caught before execution, making the shell robust against invalid commands.

2. Pipeline Execution (`executor.c`)

The executor handles process creation, pipes, and redirections. Each command in a pipeline is executed in a child process created with `fork()`. For example, to set up input/output for a command:

```
if (cmd->input_file) {
    int fd = open(cmd->input_file, O_RDONLY);
    if (fd < 0) {
        perror("Error opening input file");
        exit(EXIT_FAILURE);
    }
    dup2(fd, STDIN_FILENO);
    close(fd);
}
```

Pipes are created for multi-command pipelines so the output of one command feeds into the next:

```
int pipes[pipeline->command_count - 1][2];
for (int i = 0; i < pipeline->command_count - 1; i++) {
    if (pipe(pipes[i]) == -1) {
        perror("pipe failed");
        exit(EXIT_FAILURE);
    }
}
```

Each child process uses `dup2()` to redirect input/output to the appropriate pipe ends, while the parent closes unnecessary file descriptors to prevent resource leaks. Finally, `waitpid()` ensures all child processes finish before the shell displays the next prompt:

```
for (int i = 0; i < pipeline->command_count; i++) {
    waitpid(pids[i], &status, 0);
}
```

3. Redirection Handling

All required redirection types are supported:

- **Input (`<`)**: Reads from a file instead of standard input.
- **Output (`>`)**: Writes command output to a file.
- **Error (`2>`)**: Redirects error messages to a file.

Each redirection is safely implemented using `dup2()` and proper file descriptor management to prevent leaks.

4. Edge Case Handling

The shell includes detailed handling for multiple edge cases to emulate a real Unix shell:

1. **Empty Commands / Trailing Pipes**
   Detected during parsing using checks like:

```
if (token_count == 0 || strcmp(tokens[token_count - 1], "|") == 0) {
    fprintf(stderr, "Error: Invalid command.\n");
    return NULL;
}
```

6. **Missing Files for Redirection**
   Handled by validating that a filename exists after the redirection symbol:

```
if (i + 1 >= token_count) {
    fprintf(stderr, "Error: Missing output file.\n");
    return NULL;
}
```

11. **Invalid Commands**
    If `execvp()` fails, the child process reports the error and exits gracefully:

```
execvp(cmd->args[0], cmd->args);
perror("Command execution failed");
exit(EXIT_FAILURE);
```

15. **Large or Complex Input**
    Dynamic input buffers and token arrays ensure that long commands, multiple spaces, or combined redirections and pipelines are safely handled.

5. Memory Management

Memory is dynamically allocated for commands, arguments, pipelines, and strings. Cleanup functions prevent leaks:

```
void free_command(Command *cmd) {
    for (int i = 0; i < cmd->arg_count; i++) {
        free(cmd->args[i]);
    }
    free(cmd->args);
    free(cmd->input_file);
    free(cmd->output_file);
    free(cmd->error_file);
}

void free_pipeline(Pipeline *pipeline) {
    for (int i = 0; i < pipeline->command_count; i++) {
        free_command(&pipeline->commands[i]);
    }
    free(pipeline->commands);
}
```

Recursive cleanup ensures that nested structures are properly freed, keeping the shell stable during long sessions.

Execution Instructions

Compilation

````
The program builds using a Makefile with separate compilation and linking stages:
```makefile
# Makefile for MyShell Project
CC = gcc
CFLAGS = -Wall -Wextra -std=c99 -pedantic -g
TARGET = myshell
SOURCES = main.c input.c parser.c executor.c memory.c utils.c
OBJECTS = $(SOURCES:.c=.o)
HEADERS = shell.h

# Default target
all: $(TARGET)

# Build the executable by linking object files
$(TARGET): $(OBJECTS)
  $(CC) $(CFLAGS) -o $(TARGET) $(OBJECTS)

# Compile source files to object files with dependency on header
%.o: %.c $(HEADERS)
  $(CC) $(CFLAGS) -c $< -o $@

# Clean build artifacts
clean:
  rm -f $(OBJECTS) $(TARGET)

.PHONY: all clean
````

The Makefile uses pattern rules to automatically compile each .c file into a .o object file, then links all object files together. This two-stage approach improves build efficiency when you modify a single source file, only that file needs to be recompiled rather than the entire project.

To build and run:

make clean && make

./myshell
````

Testing

Category 1: Single Commands

Test Commands:

$ ls

$ ls -l

Screenshot 1: Single Commands

```
khalaasiyah@Khaleeqas-MacBook-Air custom-shell-implementation % ./myshell
MyShell Custom Shell Implementation
Type 'exit' to quit

$ ls
executor.c      input.txt       memory.c        parser.c        utils.c
executor.o      main.c          memory.o        parser.o        utils.o
input.c         main.o          myshell         README.md
input.o         Makefile        names.txt       shell.h
$ ls -l
total 296
-rw-r--r--@ 1 khalaasiyah  staff  17977 Sep 27 16:59 executor.c
-rw-r--r--  1 khalaasiyah  staff   8912 Oct  3 19:59 executor.o
-rw-r--r--@ 1 khalaasiyah  staff   1785 Sep 27 16:59 input.c
-rw-r--r--  1 khalaasiyah  staff   3496 Oct  3 19:59 input.o
-rw-r--r--  1 khalaasiyah  staff     24 Oct  3 19:58 input.txt
-rw-r--r--@ 1 khalaasiyah  staff   2621 Sep 27 16:59 main.c
-rw-r--r--  1 khalaasiyah  staff   3720 Oct  3 19:59 main.o
-rw-r--r--@ 1 khalaasiyah  staff   3637 Sep 20 18:08 Makefile
-rw-r--r--@ 1 khalaasiyah  staff   2998 Sep 27 16:58 memory.c
-rw-r--r--  1 khalaasiyah  staff   3368 Oct  3 19:59 memory.o
-rwxr-xr-x  1 khalaasiyah  staff  37368 Oct  3 19:59 myshell
-rw-r--r--  1 khalaasiyah  staff     21 Oct  3 19:58 names.txt
-rw-r--r--@ 1 khalaasiyah  staff   9454 Sep 27 16:59 parser.c
-rw-r--r--  1 khalaasiyah  staff   8520 Oct  3 19:59 parser.o
-rw-r--r--@ 1 khalaasiyah  staff    266 Sep 20 18:18 README.md
-rw-r--r--@ 1 khalaasiyah  staff   1616 Sep 20 18:09 shell.h
-rw-r--r--@ 1 khalaasiyah  staff   3277 Sep 27 16:58 utils.c
-rw-r--r--  1 khalaasiyah  staff   3912 Oct  3 19:59 utils.o
$
```

Testing single commands without arguments (ls) and with arguments (ls -l). Both commands executed successfully and displayed the expected output.

Explanation:

- Test 1 (ls): Successfully listed all files in the current directory
- Test 2 (ls -l): Correctly displayed detailed file information including permissions, owner, size, and modification date.

Category 2: Input, Output, and Error Redirection

Test Commands:

$ sort < names.txt

$ ls -l > output.txt

$ cat output.txt

$ invalidcommand123 2> error.log

$ cat error.log

Screenshot 2: Redirection Tests

```
$ sort < names.txt
Alice
Bob
Chris
Zack
$ ls -l > output.txt
$ cat output.txt
total 296
-rw-r--r--@ 1 khalaasiyah  staff  17977 Sep 27 16:59 executor.c
-rw-r--r--  1 khalaasiyah  staff   8912 Oct  3 19:59 executor.o
-rw-r--r--@ 1 khalaasiyah  staff   1785 Sep 27 16:59 input.c
-rw-r--r--  1 khalaasiyah  staff   3496 Oct  3 19:59 input.o
-rw-r--r--  1 khalaasiyah  staff     24 Oct  3 19:58 input.txt
-rw-r--r--@ 1 khalaasiyah  staff   2621 Sep 27 16:59 main.c
-rw-r--r--  1 khalaasiyah  staff   3720 Oct  3 19:59 main.o
-rw-r--r--@ 1 khalaasiyah  staff   3637 Sep 20 18:08 Makefile
-rw-r--r--@ 1 khalaasiyah  staff   2998 Sep 27 16:58 memory.c
-rw-r--r--  1 khalaasiyah  staff   3368 Oct  3 19:59 memory.o
-rwxr-xr-x  1 khalaasiyah  staff  37368 Oct  3 19:59 myshell
-rw-r--r--  1 khalaasiyah  staff     21 Oct  3 19:58 names.txt
-rw-r--r--  1 khalaasiyah  staff      0 Oct  3 20:02 output.txt
-rw-r--r--@ 1 khalaasiyah  staff   9454 Sep 27 16:59 parser.c
-rw-r--r--  1 khalaasiyah  staff   8520 Oct  3 19:59 parser.o
-rw-r--r--@ 1 khalaasiyah  staff    266 Sep 20 18:18 README.md
-rw-r--r--@ 1 khalaasiyah  staff   1616 Sep 20 18:09 shell.h
-rw-r--r--@ 1 khalaasiyah  staff   3277 Sep 27 16:58 utils.c
-rw-r--r--  1 khalaasiyah  staff   3912 Oct  3 19:59 utils.o
$ invalidcommand123 2> error.log
$ cat error.log
Command not found: invalidcommand123
$
```

Testing input redirection (<), output redirection (>), and error redirection (2>). All redirection operations worked correctly with proper file descriptor management.

Explanation:

- Test 3 (Input Redirection): Successfully read from names.txt and sorted the contents alphabetically using the < operator
- Test 4 (Output Redirection): Correctly redirected standard output to output.txt using the > operator, with no terminal output displayed
- Test 5 (Error Redirection): Properly captured error messages to error.log using 2> operator, demonstrating correct STDERR handling.

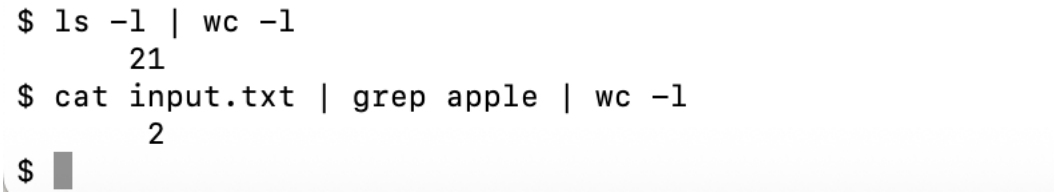
Category 3: Pipes (5 points)

Test Commands:

$ ls -l | wc -l

$ cat input.txt | grep apple | wc -l

Screenshot 3: Pipe Operations



Testing simple two-command pipe and multi-command pipe chain. Pipes correctly connected standard output of one command to standard input of the next.

Explanation:

- Test 6 (Simple Pipe): Successfully piped ls -l output to wc -l, demonstrating proper pipe() system call implementation and process communication
- Test 7 (Multi-Pipe Chain): Correctly executed three-command pipeline, showing support for n number of pipes as required. Each process correctly inherited file descriptors and communicated through pipes.

Category 4: Compound Commands (10 points)

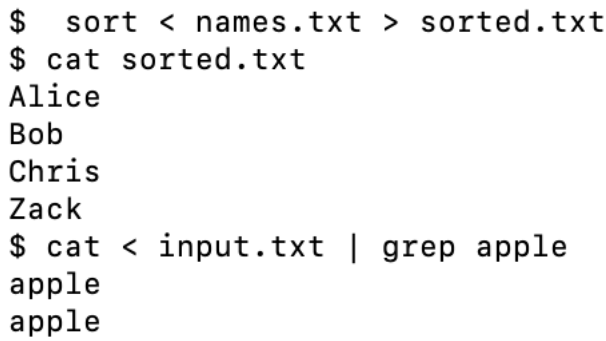
Test Commands - Part 1:

$ sort < names.txt > sorted.txt

$ cat sorted.txt

$ cat < input.txt | grep apple

Screenshot 4: Compound Commands Part 1



Testing compound commands combining input/output redirection and pipes. Commands demonstrate correct handling of multiple redirections and pipe operations together.

Explanation:

- Test 8 (Input + Output Redirection): Successfully combined < and > operators in a single command, demonstrating proper dup2() usage for both input and output file descriptors
- Test 9 (Input + Pipe): Correctly read from file using < and piped the output to grep, showing proper coordination between redirection and pipe operations

Test Commands - Part 2:

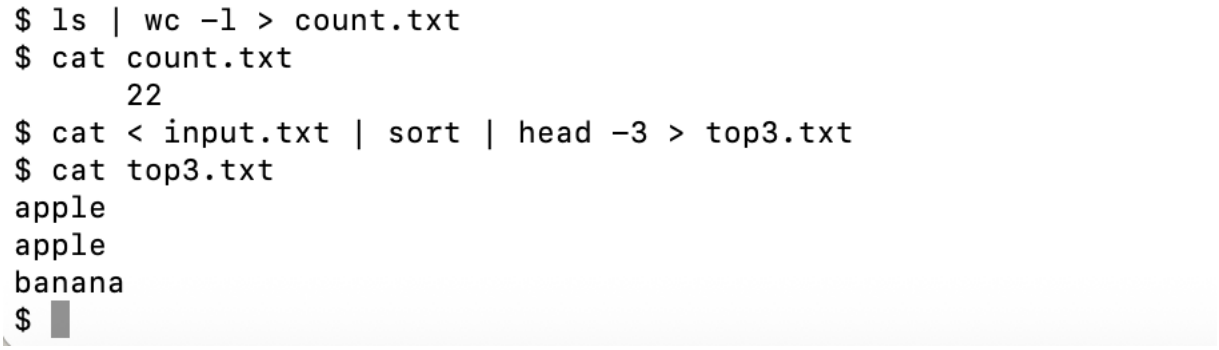
$ ls | wc -l > count.txt

$ cat count.txt

$ cat < input.txt | sort | head -3 > top3.txt

$ cat top3.txt

Screenshot 5: Compound Commands Part 2



Testing advanced compound commands including pipe with output redirection and complex multi-pipe with input/output redirection. All compound operations executed successfully.

Explanation:

- Test 10 (Pipe + Output Redirection): Successfully piped command output and redirected final result to file, demonstrating proper file descriptor management in pipeline with output redirection
- Test 11 (Complex Multi-Pipe with Redirections): Executed most complex compound command combining input redirection (<), multiple pipes (|), and output redirection (>). This demonstrates full implementation of all required compound combinations with proper process creation, pipe setup, and file descriptor duplication.

Category 5: Error Handling (3 points)

Test Commands:

$ cat <

$ echo hello >

$ ls |

$ thisdoesnotexist999

Screenshot 6: Error Handling

```
$ cat <
Error: Input file not specified.
$ echo hello >
Error: Output file not specified.
$  ls |
Error: Empty command.
$ thisdoesnotexist999
Command not found: thisdoesnotexist999
$
```

Testing error detection and handling for various invalid commands and syntax errors. All errors were caught and reported with appropriate messages without crashing the shell.

Explanation:

- Test 12 (Missing Input File): Parser correctly detected missing filename after < operator and displayed appropriate error message
- Test 13 (Missing Output File): Parser correctly detected missing filename after > operator and displayed appropriate error message
- Test 14 (Missing Command After Pipe): Parser correctly detected incomplete pipe command and displayed appropriate error message
- Test 15 (Invalid Command): Shell properly handled execvp() failure and displayed command not found error message

All error cases were handled gracefully without shell crashes or hangs, demonstrating robust error handling throughout the parsing and execution phases.

Exit Functionality

Test Command:

```
$ exit
```

Screenshot 7: Exit Command

```
khalaasiyah@Khaleeqas-MacBook-Air custom-shell-implementation % ./myshell
MyShell Custom Shell Implementation
Type 'exit' to quit

$ exit
Shell terminated.
khalaasiyah@Khaleeqas-MacBook-Air custom-shell-implementation %
```

Testing shell exit functionality. The exit command successfully terminated myshell and returned to the system shell.

Explanation: The exit command was properly recognized and terminated the shell cleanly, returning control to the parent system shell with no hanging processes.

Challenges

1. **Pipe File Descriptor Management**:
   At first I left file descriptors open, which caused problems. I solved this by carefully closing unused ones in both parent and child processes.
2. **Process Synchronization**:
   Commands sometimes returned control before finishing. I fixed this by using `waitpid()` on every child process.
3. **Parsing Complexity**:
   Mixing pipes and redirections was tricky. I solved this by first splitting on pipes, then processing each command for arguments and redirections.
4. **Memory Leaks**:
   Because I used dynamic allocation, I needed strict cleanup. I added dedicated `free()` functions.
5. **Error Validation**:
   Designing checks for invalid commands took time, but it made the shell much more stable.

Division of Tasks

This project was completed individually. I was responsible for every part, including:

- Designing the system architecture and file structure.
- Writing the main control loop and user input functions.
- Implementing the parser, executor, and memory management.
- Creating utility functions for error handling.
- Testing the shell extensively across multiple categories.
- Writing documentation and this report.

References

1. Silberschatz, Abraham, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*. 10th ed., Wiley, 2018.
2. Stevens, W. Richard, and Stephen A. Rago. *Advanced Programming in the UNIX Environment*. 3rd ed., Addison-Wesley, 2013.
3. "access(2) - Linux Manual Page." *Linux Programmer's Manual*, https://man7.org/linux/man-pages/man2/access.2.html. Accessed 3 Oct. 2025.
4. "dup2(2) - Linux Manual Page." *Linux Programmer's Manual*, https://man7.org/linux/man-pages/man2/dup2.2.html. Accessed 3 Oct. 2025.
5. "execvp(3) - Linux Manual Page." *Linux Programmer's Manual*, https://man7.org/linux/man-pages/man3/exec.3.html. Accessed 3 Oct. 2025.
6. "fork(2) - Linux Manual Page." *Linux Programmer's Manual*, https://man7.org/linux/man-pages/man2/fork.2.html. Accessed 3 Oct. 2025.
7. "pipe(2) - Linux Manual Page." *Linux Programmer's Manual*, https://man7.org/linux/man-pages/man2/pipe.2.html. Accessed 3 Oct. 2025.
8. "waitpid(2) - Linux Manual Page." *Linux Programmer's Manual*, https://man7.org/linux/man-pages/man2/waitpid.2.html. Accessed 3 Oct. 2025.
9. "String and Array Utilities." *The GNU C Library Reference Manual*, Free Software Foundation, https://www.gnu.org/software/libc/manual/html_node/String-and-Array-Utilities.html. Accessed 3 Oct. 2025.